pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Treasury Curve Interpolation

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

A yield at any maturity: two curves, two jobs

A bond with 6.4 years left needs the Treasury yield at 6.4 years. Treasury publishes a short list of tenors. Every maturity in between is read off a line drawn through them.

Real: Treasury par curve, 30 Sep 2026

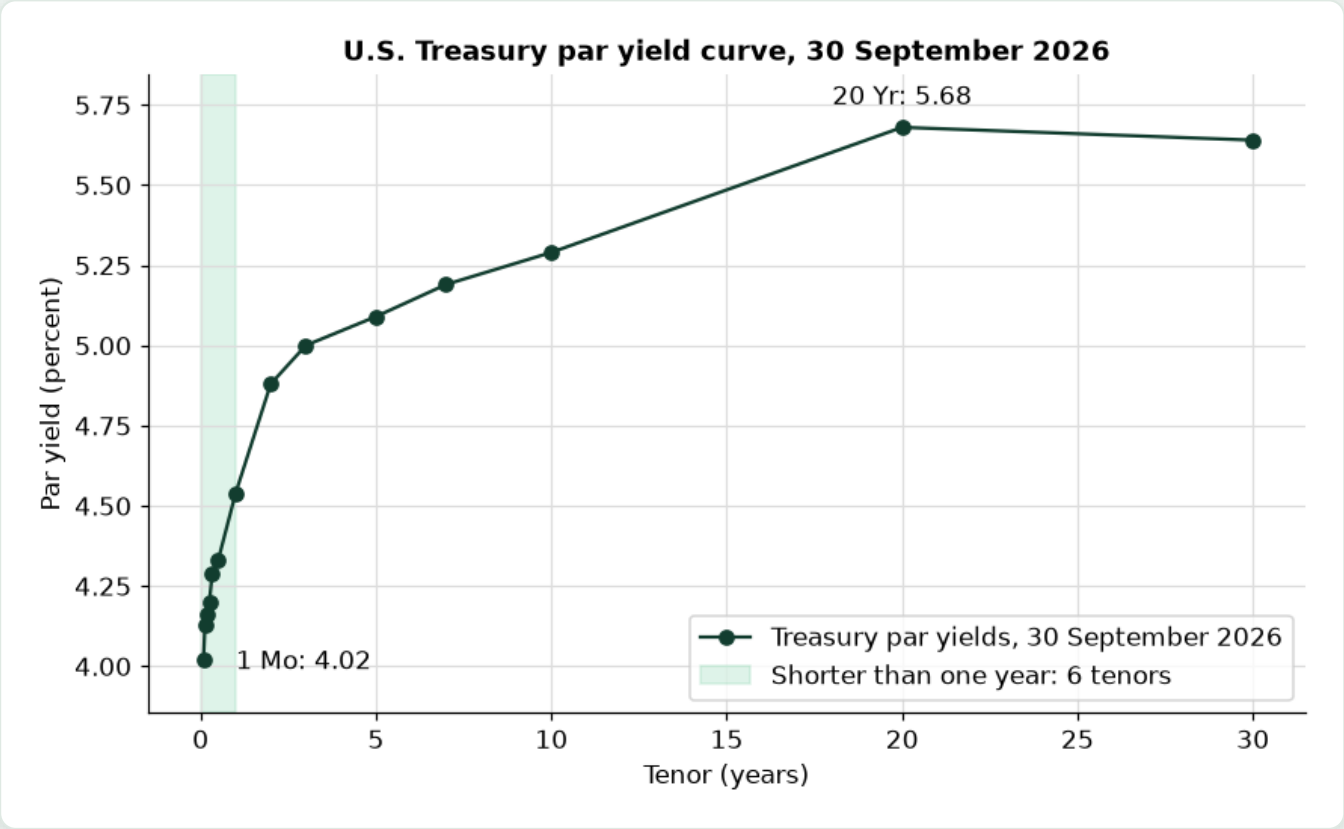
From Treasury's published file. Teaches the method. Described only, never used for a spread: the holdings were not built on it.

Invented: ten tenors, the holdings curve

The curve the holdings file's yields were built on. Days 12 to 14 measure every spread from it.

Day 11	Day 12	Day 13	Day 14	Day 15
The curve	Spread	Spread duration, DTS	The book, scenarios	Capstone

A curve is tenors and yields



Label	Years	Yield
1 Mo	0.0833	4.02
1.5 Month	0.125	4.13
3 Mo	0.25	4.20
6 Mo	0.5	4.33
1 Yr	1	4.54
10 Yr	10	5.29
30 Yr	30	5.64

Labels are text. Months are twelfths of a year. 6 of the 14 tenors are under a year.

Source: U.S. Department of the Treasury, Daily Treasury Par Yield Curve Rates. Described only.

A straight line between two tenors

```
# 6.4 years lies between 5 Yr and 7 Yr
weight = (6.4 - 5) / (7 - 5)        # 0.7
left, right = 5.09, 5.19
left + weight * (right - left)      # 5.1600
```

Linear interpolation: a weighted average of the two neighbours.

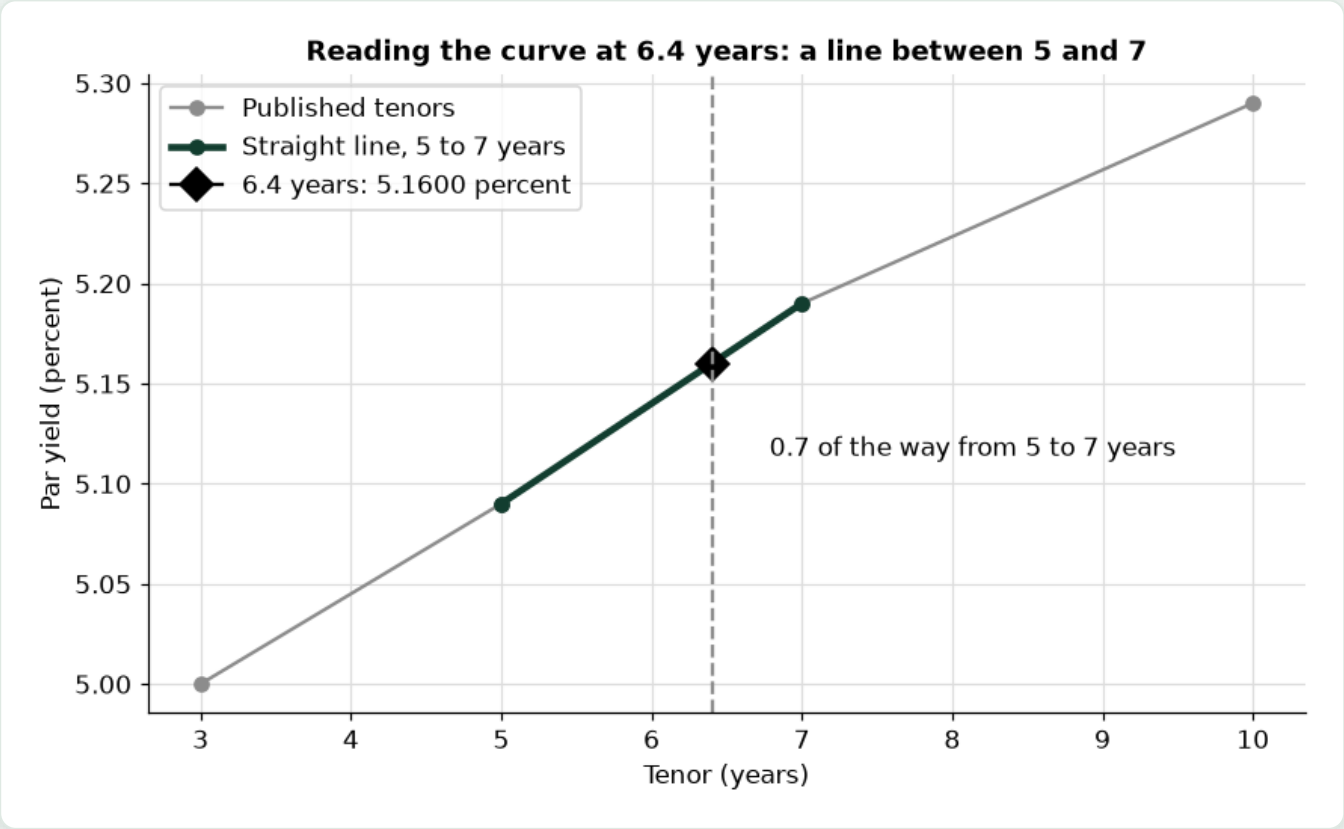
Excel side by side (tested in Excel)

`=FORECAST.LINEAR(6.4,B2:B3,A2:A3)`	5.1600
`=B2+(B3-B2)*(6.4-A2)/(A3-A2)`	5.1600
`=FORECAST.LINEAR(6.4,A2:A3,B2:B3)`	31.20

5 and 7 in A2:A3, the yields in B2:B3. Yields come before tenors: swapped, Excel returns a number with no error.

The bracket: =MATCH(6.4, tenors, 1) returns 10, the position of 5 years; INDEX turns it back into 5 and 7.

Reading 6.4 years off the line




```
5.09 + 0.7 x (5.19 - 5.09) = 5.1600
```

The thick line is the only part of the curve the answer uses. The diamond sits 0.7 of the way from the 5-year point to the 7-year point.

Between two tenors the curve is whatever straight line you draw. Section 11.6 measures what that choice costs.

interpolate_yield: three decisions first

```
# flat beyond the ends
if tenor_years <= curve_tenors[0]:
    return curve_yields_pct[0]
# find the two tenors around tenor_years
i = 1
while curve_tenors[i] < tenor_years:
    i += 1
# move along the straight line between them
return left_yield + slope * (tenor_years - left_tenor)
```

Units

Tenors in years, yields in percent, the result in percent.

Flat ends

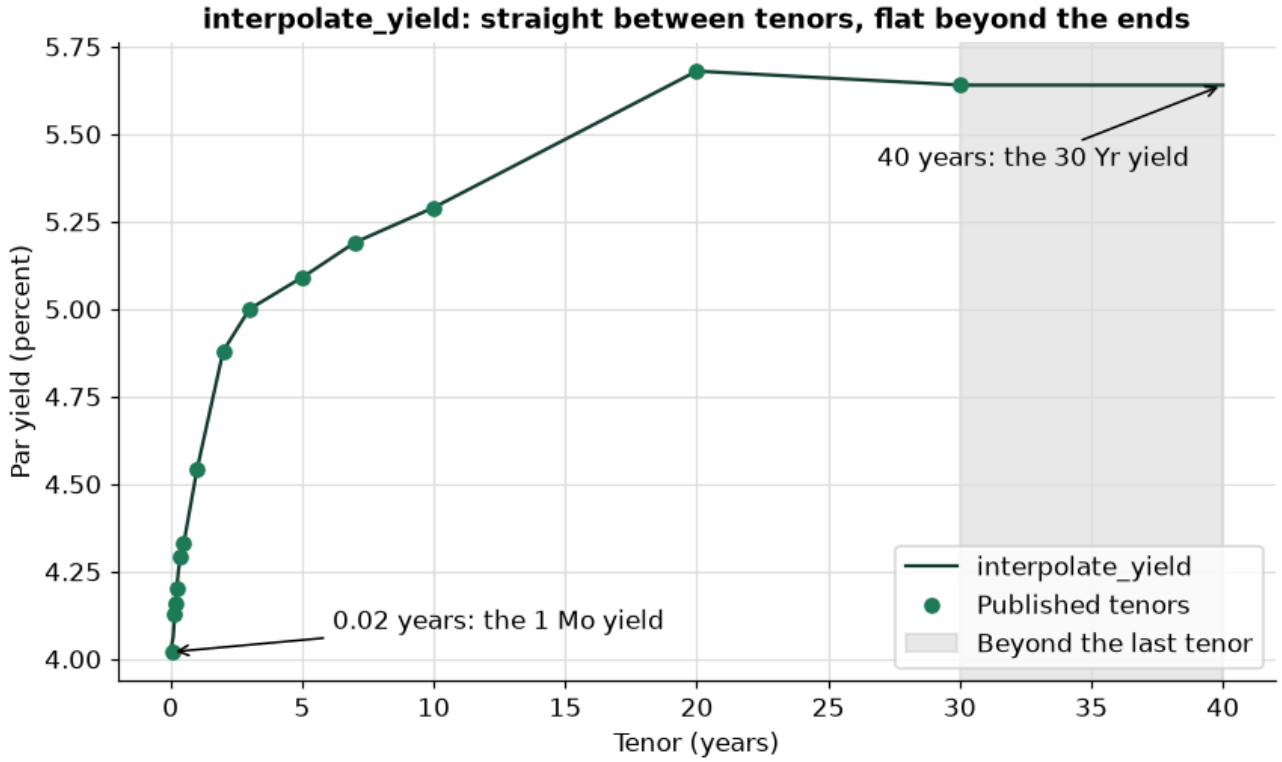
The first yield below the first tenor, the last beyond the last, as numpy.interp.

The check

Tenors must ascend, or ValueError: a jumbled curve never answers.

Four tests on the invented curve: on a tenor, against numpy.interp, flat ends, raises. 32 passed.

Flat beyond the ends



4.02

at 0.02 years: the 1 Mo yield

5.64

at 40 years: the 30 Yr yield

Between the dots the line is straight. Past the last dot it runs level, and before the first dot it would too.

The function never invents a slope it was not given.

numpy.interp: the same line, no check

Tenor	interpolate_yield	np.interp
0.02	4.0200	4.0200
2.5	4.9400	4.9400
6.4	5.1600	5.1600
15	5.4850	5.4850
40	5.6400	5.6400

Every tenor agrees, both ends included: numpy is flat there too.

A curve typed out of order: 1, 3, 2

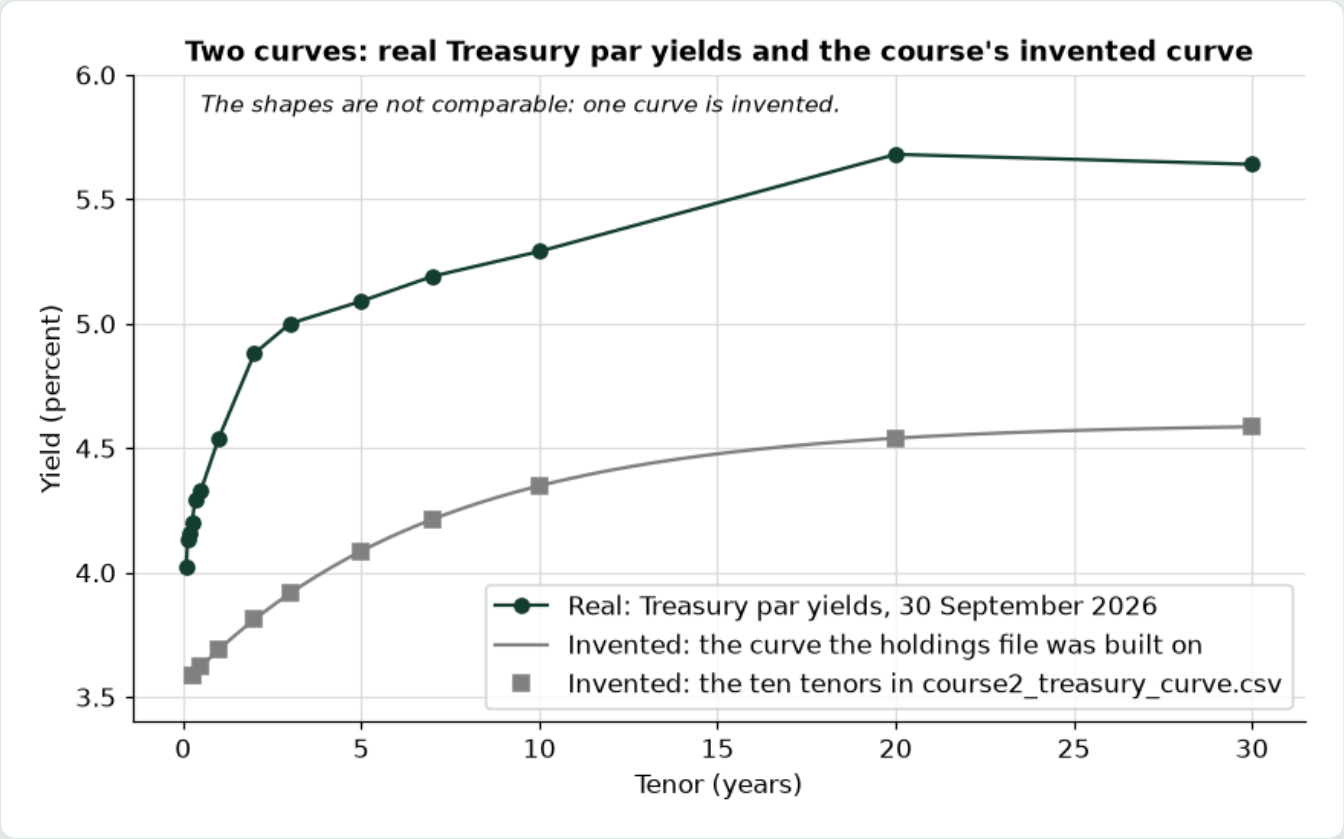
```
np.interp(4, [1, 3, 2], ...)  ->  3.8
```

```
interpolate_yield  ->  ValueError
```

numpy's documentation: the tenors are expected to increase, this is not enforced, and if they do not, the results are meaningless.

The check belongs to whoever calls numpy. Our function makes it.

The curve the holdings were built on



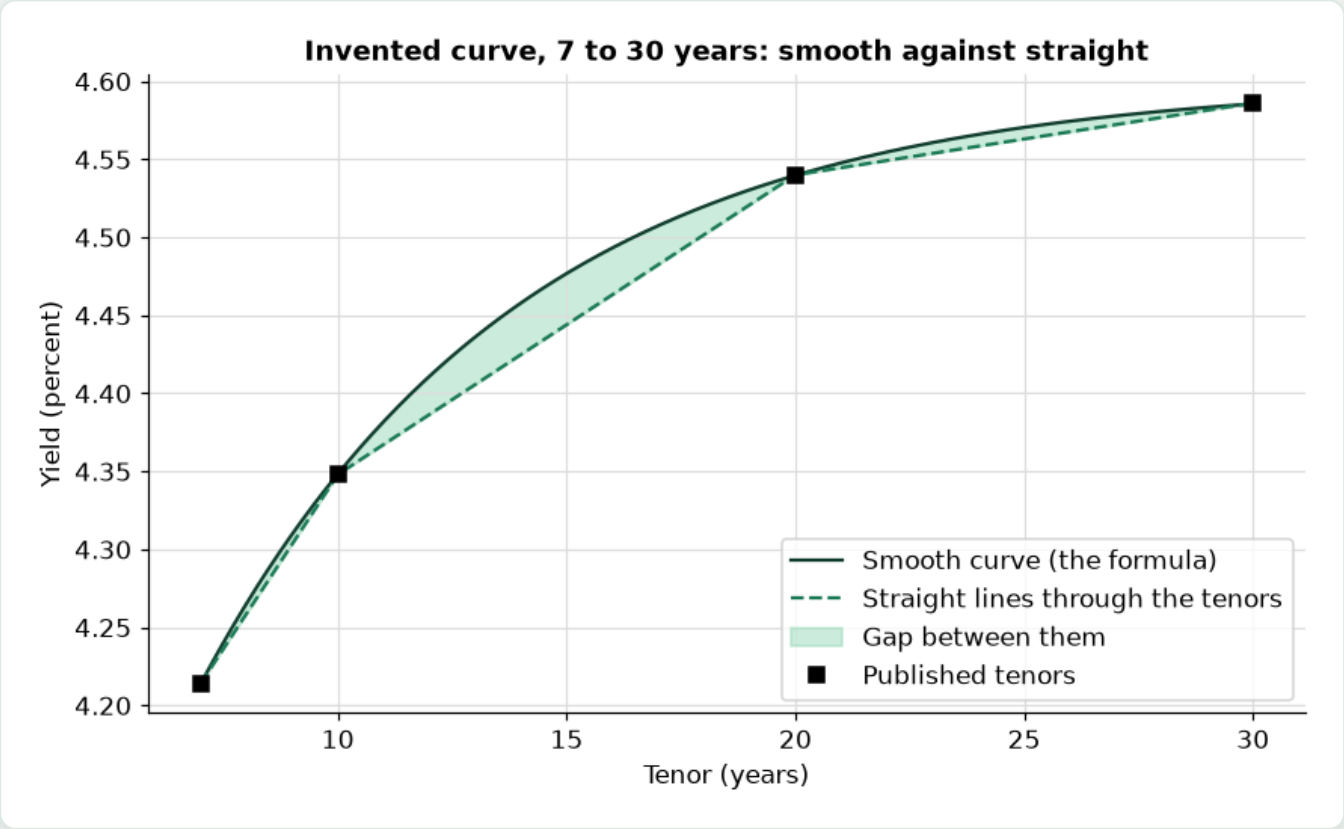

```
3.55 + 1.05 x (1 - exp(-T / 7))
```

Ten tenors, rounded to 3 decimals: 3.587 percent at 0.25 years to 4.586 at 30.

The shapes are not comparable: one curve is invented.

Written to give synthetic bonds believable yields, never fitted to Treasury's figures. Days 12 to 14 use it.

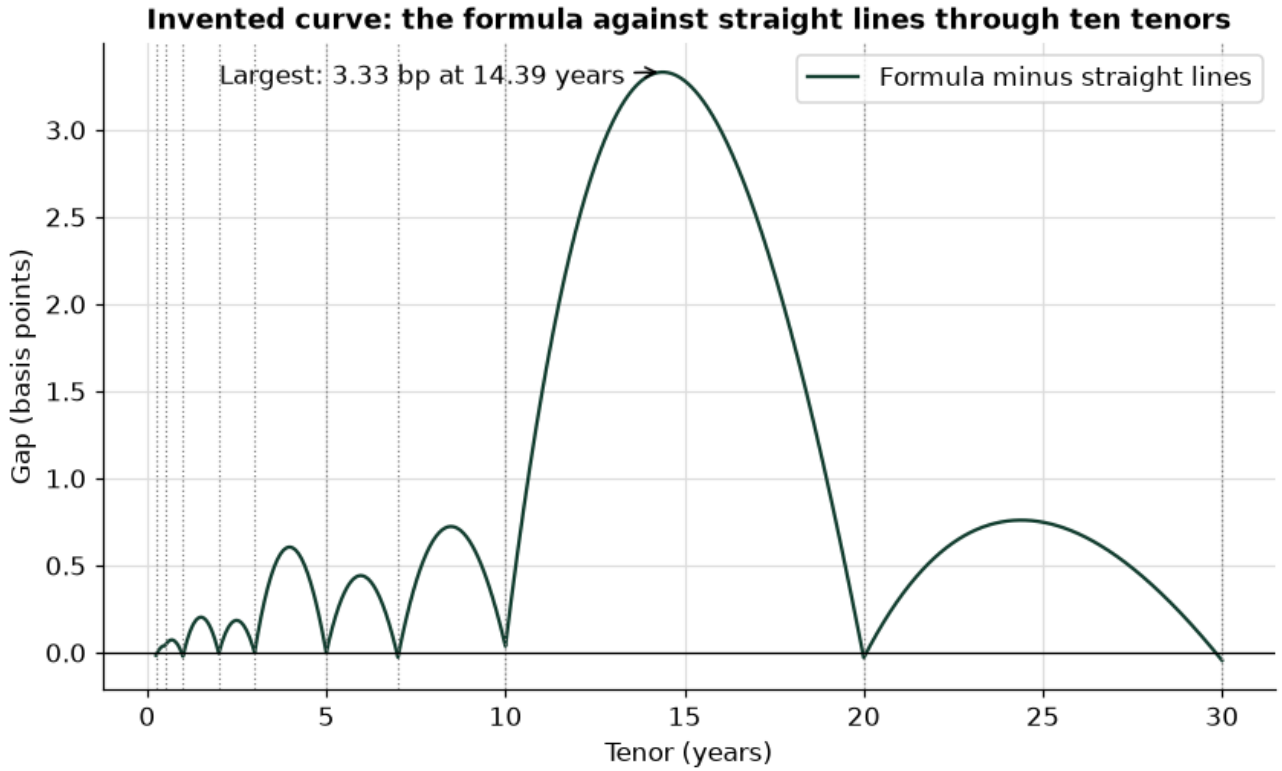
Straight lines under a bending curve



The solid line is the formula. The dashed line is interpolate_yield through the ten tenors. The squares are the only points both share.

The shaded area is the gap: the curve bulges above the straight line, most between 10 and 20 years.

How big the gap is



3.33 bp

largest gap, at 14.39 years

1.01 bp

average over the grid

Zero at each tenor, a hump between each pair, largest where the tenors are far apart and the curve still bends.

The interpolation choice moves the number. Day 12: what it does to each spread.

Today's Excel, every formula tested

Job	Excel (tested), curve in A2:B15, tenor in D1	Returns
Line through two points	`=FORECAST.LINEAR(6.4,B2:B3,A2:A3)`	5.1600
The bracket's position	`=MATCH(D1,A2:A15,1)`	10
The bracket's tenor	`=INDEX(A2:A15,MATCH(D1,A2:A15,1))`	5
Inside the curve	`FORECAST.LINEAR on the INDEX/MATCH bracket`	5.1600
Below the first tenor	`the same formula, 0.02 in D1`	#N/A
At or beyond the last	`the same formula, 30 or 35 in D1`	#REF!
The chart	`A1:B15, Insert, Scatter with Straight Lines`	one series

Use the formula strictly inside the curve. Chart a curve as a Scatter: a Line chart spaces the tenors evenly.

Commit the data the tests rely on

First today's work: interpolate_yield and its four tests. Then the curve file the tests' ten points come from, so the source of every number sits in the same record.

The commands

```
git add course2_treasury_curve.csv
git commit -m "Add the invented curve ..."
git log --stat -1
```

git log --stat -1, the end of it

```
    Add the invented curve file ...
course2_treasury_curve.csv | 11 ++++++++++
1 file changed, 11 insertions(+)
```

-1: only the last commit. --stat: the files it changed, and how many lines in each.

Bring it home: paste, copy, test, commit

1

Your folder

Set-Location to my_bondmath, in your projects folder.

2

Paste

Today's function and four tests at the end of your files, without the %%writefile line.

3

Copy

Copy-Item the curve file from the course repo's data folder.

4

Test

python -m pytest -q: 32 passed, more with your exercise functions.

5

Commit

Commit the code, then the curve file, then git log --stat -1.

```
Copy-Item "$HOME\projects\python-fixed-income\data\course2_treasury_curve.csv" .
```

In Colab: download bondmath.py, test_bondmath.py, and course2_treasury_curve.csv from the work folder.

The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.

1

Docstring

Units, convention, and the Excel formula: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert against Excel's reference rows. Run it: it fails.

4

Fix

Find the convention it broke, fix it, run the test again.

Today: the "3 Mo" label is read as 3 years, so the month tenors land among the year tenors.

Code that runs is not the same as code that is right.

What you can do now

- Read a curve as tenors and yields; turn Treasury's labels into years
- Interpolate by hand; find the bracket with a loop or MATCH
- interpolate_yield: straight lines, flat ends, a stop on a jumbled curve
- Match numpy.interp and Excel's FORECAST.LINEAR; chart a curve as a Scatter
- Tell the real curve from the invented one; measure the straight-line gap

Week 3, Day 2 Spread over the curve: each bond's yield over the invented curve at its own maturity